Informe de Trabajo Práctico 1

1. Carátula

- Título del Trabajo: Aplicaciones de TADs
- Integrantes del equipo:
  - Daiana Nicole DINAMARCA
  - Julian Bautista NEME Ferrari del Sel
- Fecha de entrega: 04/05/2026
- Link del repositorio: https://github.com/JuliFdS/AyED2026c1-Neme-Dinamarca.git

2. Introducción

El presente trabajo aborda la implementación de una estructura de datos lineal personalizada (Lista Doble Enlazada) y su aplicación en la simulación del juego de cartas “Guerra” (Problema 2). Asimismo, se realiza un estudio sobre la eficiencia de distintos algoritmos de ordenamiento (Problema 3) aplicados sobre estructuras de datos nativas, con el fin de validar conceptos de complejidad computacional mediante el análisis de tiempos de ejecución reales y su contraste con la teoría asintótica.

3. Desarrollo de los problemas

3.1. Problema N° 1: TAD Lista Doblemente Enlazada

A. Análisis y planteo

La resolución del Problema 1 se centró en la creación de un Tipo Abstracto de Datos (TAD) que garantice la manipulación eficiente de datos genéricos. A diferencia de las listas contiguas (como los arrays de Python), la **Lista Doblemente Enlazada (LDE)** utiliza nodos independientes vinculados bidireccionalmente, lo que permite una flexibilidad total en la gestión de memoria.

Algunas consideraciones:

- Para satisfacer el requerimiento de complejidad O(1) en ambos extremos, se planteó una estructura con referencias directas a _cabeza y _cola. Esto es crítico para operaciones de "extracción" al final de la lista, que en una lista simplemente enlazada requerirían un recorrido lineal O(n).
- Se adoptó el uso de Decoradores (@property) para gestionar los atributos privados. Esto no solo cumple con la consigna, sino que permite validar el estado de la estructura (como el tamaño) de manera centralizada.
- Se planteó que las búsquedas internas no deben ser siempre desde el inicio. Al conocer el tamaño actual, fue posible implementar una lógica de "extremo más cercano" para optimizar operaciones en posiciones arbitrarias.

B. Implementación / Resolución

La implementación técnica se divide en la clase Nodo y la clase contenedora ListaDobleEnlazada.

```
class Nodo:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None
        self.anterior = None

class ListaDobleEnlazada:
    """
    Implementación del TAD Lista Doblemente Enlazada para el Problema 1.
    Permite almacenar elementos comparables y respeta la especificación lógica.
    """

    def __init__(self):
        """
        Consigna: El inicializador __init__ debe crear una lista originalmente vacía.
        """
        self._cabeza = None
        self._cola = None
        self._tamanio = 0
```

Iterabilidad mediante Generadores: Se implementó __iter__ utilizando la palabra clave yield. Esto permite que la lista sea recorrida de forma eficiente y perezosa (lazy evaluation) con ciclos for, cumpliendo con el protocolo de iteración de Python sin sobrecargar la memoria.

```
def __iter__(self):
    """__iter__(): Permite que la lista sea recorrida con un ciclo for."""
    actual = self.cabeza
    while actual:
        yield actual.dato
        actual = actual.siguiente
```

Copia Eficiente O(n): El método copiar() se resolvió mediante un bucle while que recorre la lista original de forma secuencial. Al utilizar internamente agregar_al_final() (que es O(1) gracias al puntero _cola), el costo total se mantiene lineal. Se evitó explícitamente el uso de métodos de inserción por índice que degradarían la complejidad a O(n^2).

```python
def copiar(self):
    """
    copiar(): Realiza una copia elemento a elemento y devuelve la copia.
    Consigna: Verificar que el orden sea O(n) y no O(n^2).
    """
    nueva_lista = ListaDobleEnlazada()
    actual = self.cabeza
    while actual:
        # Como agregar_al_final es O(1), el bucle total es O(n)
        nueva_lista.agregar_al_final(actual.dato)
        actual = actual.siguiente
    return nueva_lista
```

Inversión de Punteros (In-Place): El método invertir() se implementó sin crear una lista nueva. La lógica consiste en iterar por cada nodo intercambiando sus referencias siguiente y anterior. Al finalizar, se reasignan la cabeza y la cola de la clase.

```python
def invertir(self):
    """invertir(): Invierte el orden de los elementos de la lista."""
    if self.esta_vacia() or self.tamanio == 1:
        return

    actual = self.cabeza
    self.cola = actual
    temporal = None

    while actual:
        temporal = actual.anterior
        actual.anterior = actual.siguiente
        actual.siguiente = temporal
        actual = actual.anterior # Avanza usando el puntero anterior (que ahora es el original siguiente)

    if temporal:
        self.cabeza = temporal.anterior
```

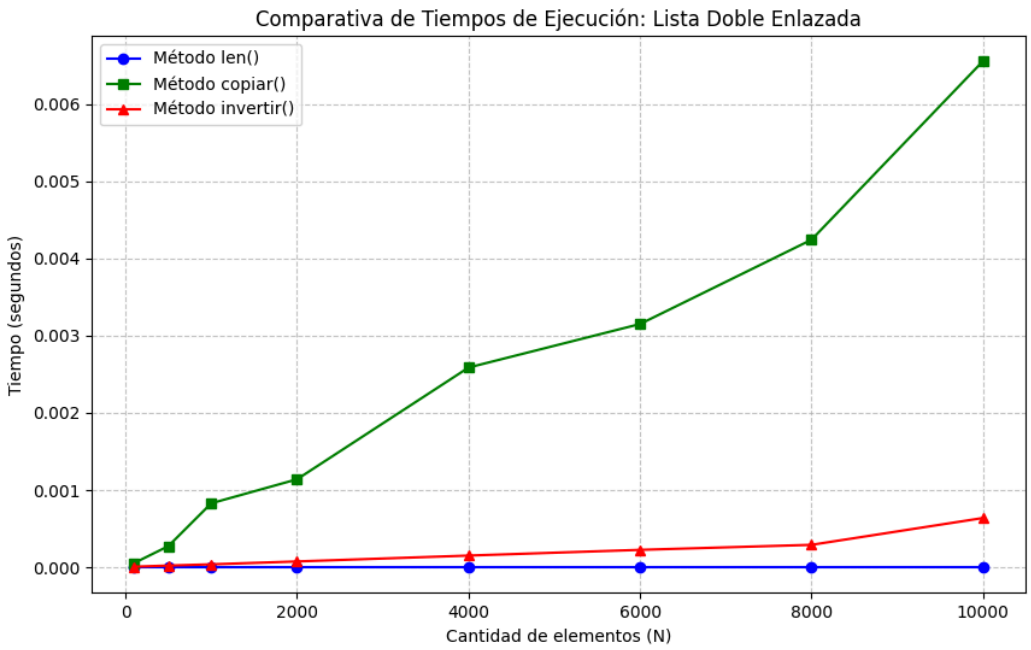
Uso de Métodos Auxiliares: Se destaca el método _get_nodo(posicion), el cual divide el espacio de búsqueda. Si la posicion está en la segunda mitad de la lista, el recorrido se inicia desde la cola hacia atrás, reduciendo las iteraciones necesarias a un máximo de n/2.

C. Resultados y pruebas

Los datos obtenidos experimentalmente mediante el script de benchmarking validan las hipótesis de complejidad asintótica planteadas en el diseño. A continuación, se analizan los comportamientos observados en la gráfica:

- **Método len() (O(1)):** La gráfica muestra una línea horizontal prácticamente constante y cercana al eje X. Esto confirma que la implementación es eficiente, ya que no recorre la lista para contar los elementos, sino que accede directamente al atributo self._tamanio. El tiempo de respuesta es independiente de N.

- **Método invertir() (O(n)):** Se observa un crecimiento lineal proporcional al tamaño de la lista. Técnicamente, este método tiene una pendiente baja porque solo realiza **reasignación de punteros** (referencias en memoria), lo cual es una operación extremadamente rápida para el procesador. No se crea memoria nueva, solo se "re-enlazan" los nodos existentes.
- **Método copiar() (O(n)):** Aunque su complejidad también es lineal, presenta una pendiente más pronunciada que invertir(). Esto se justifica porque copiar() realiza un trabajo doble: además de recorrer los N elementos de la lista original, debe **instanciar N nuevos objetos de la clase Nodo** y asignar memoria para ellos. La creación de objetos es más costosa en tiempo que el simple intercambio de punteros.



En conclusión, las mediciones reales coinciden con el análisis a priori. Se logró optimizar copiar() para evitar una degradación a O(n^2) y se garantizó que la consulta de tamaño sea instantánea.

3.2. Problema N° 2: Mazo y Juego de Guerra

A. Análisis y planteo

Para esta consigna se nos pidió modelar un mazo de cartas y la lógica de un juego de "Guerra" utilizando la estructura de datos desarrollada en el Problema 1. En otras palabras, simular un juego de cartas con lógica de confrontación y acumulación. El desafío técnico principal fue implementar un motor de juego que maneje estados de empate ("Guerra") y la transferencia masiva de cartas entre estructuras.

- **Operaciones críticas:**

- **Reparto y Juego:** Requieren extraer elementos desde la parte superior (inicio) de la estructura.
- **Recolección de botín:** Requiere insertar las cartas ganadas al final del mazo del ganador.

- **Estructura elegida:** La ListaDobleEnlazada es ideal aquí porque permite extraer de la cabeza e insertar en la cola en tiempo constante. Satisface perfectamente las necesidades por ser una estructura de Bicoda (Deque).
- **Manejo de Errores:** Se definió la excepción DequeEmptyError para gestionar el final de la partida. El motor del juego utiliza un bloque try-except para detectar cuándo un jugador no puede continuar (especialmente durante una Guerra), lo que garantiza un flujo de programa robusto y sin cierres inesperados.

B. Implementación / Resolución

La implementación se estructuró en tres módulos desacoplados:

1. **Módulo carta.py:** Se implementó la clase Carta utilizando **decoradores @property** para el estado de visibilidad, valor y palo. Para permitir comparaciones directas entre objetos (como carta1 > carta2), se sobrecargaron los métodos mágicos __gt__ (greater than) y __eq__ (equal), utilizando una función auxiliar _valor_numerico que traduce las figuras (J, Q, K, A) a valores comparables (11-14).

```
def __gt__(self, otra):
    """Compara si una carta es mayor a otra por valor numérico."""
    return self._valor_numerico() > otra._valor_numerico()
```

2. **Módulo mazo.py:** Define la clase Mazo. Esta clase no utiliza arreglos nativos de Python; en su lugar, instancia una ListaDobleEnlazada. Los métodos poner_carta_arriba y sacar_carta_arriba mapean directamente a agregar_al_inicio y extraer(0) respectivamente, asegurando la eficiencia exigida por la cátedra.

```
def poner_carta_arriba(self, carta):
    """Agrega una carta al inicio del mazo."""
    self._cartas.agregar_al_inicio(carta)
```

```
def sacar_carta_arriba(self, mostrar=False):
    """
    Extrae la carta de arriba. Lanza DequeEmptyError si está vacío.
    Si 'mostrar' es True, hace la carta visible.
    """
    if self._cartas.esta_vacia():
        raise DequeEmptyError("El mazo está vacío.")

    carta = self._cartas.extraer(0)
    if mostrar:
        carta.visible = True
    return carta
```

3. **Módulo juego_guerra.py:** Contiene la lógica del negocio. Utiliza un bucle principal que se ejecuta hasta que un mazo queda vacío o se alcanza el límite de N_TURNOS (10,000). Durante el estado de "Guerra", el motor extrae 3 cartas

ocultas de cada jugador y una cuarta visible, acumulándolas en una lista temporal antes de otorgar el pozo al ganador del turno.

```
while len(self.mazo_1) and len(self.mazo_2) and self._turno != N_TURNOS:
    try:
        if self._guerra:
            for _ in range(3):
                self._cartas_en_la_mesa.append(self.mazo_1.sacar_carta_arriba())
                self._cartas_en_la_mesa.append(self.mazo_2.sacar_carta_arriba())

        self._cartas_en_la_mesa.append(self.mazo_1.sacar_carta_arriba(mostrar=True))
        self._cartas_en_la_mesa.append(self.mazo_2.sacar_carta_arriba(mostrar=True))
```

B. Implementación / Resolución

La solución se validó mediante la ejecución del script principal (juego_guerra.py), simulando partidas con distintas semillas aleatorias (random_seed).

- **Correctitud:** El motor detecta correctamente al ganador tras el agotamiento del mazo rival.
- **Representación:** El método __str__ del Mazo fue optimizado para utilizar el iterador de la ListaDobleEnlazada mediante una expresión generadora (" ".join(...)), lo que permite visualizar el estado de la partida en consola de forma clara y eficiente.
- **Test de Cátedra:** El código pasa satisfactoriamente las pruebas unitarias de Mazo y JuegoGuerra.

3.3. Problema N° 3: Algoritmos de Ordenamiento

A. Análisis y planteo

El objetivo de este problema fue contrastar la eficiencia de tres paradigmas de ordenamiento distintos frente a la implementación nativa de Python.

- **Operaciones críticas:** En Burbuja y Quicksort, la operación crítica es la comparación e intercambio. En Radix Sort, es la distribución en buckets (conteo de frecuencias).
- **Naturaleza de los datos:** Se trabajó con listas de números enteros de 5 dígitos (rango 10,000 a 99,999). Este dato es clave para el algoritmo **Radix Sort**, cuya eficiencia depende de la cantidad de dígitos (k=5).
- **Supuestos de complejidad:** Se planteó que para N=1000, la diferencia entre un algoritmo cuadrático (O(n^2)) y uno logarítmico (O(n log n)) sería evidente, mientras que el algoritmo lineal (O(nk)) debería mostrar una estabilidad superior.

B. Implementación / Resolución

Se implementó una clase Ordenador con métodos estáticos para encapsular la lógica:

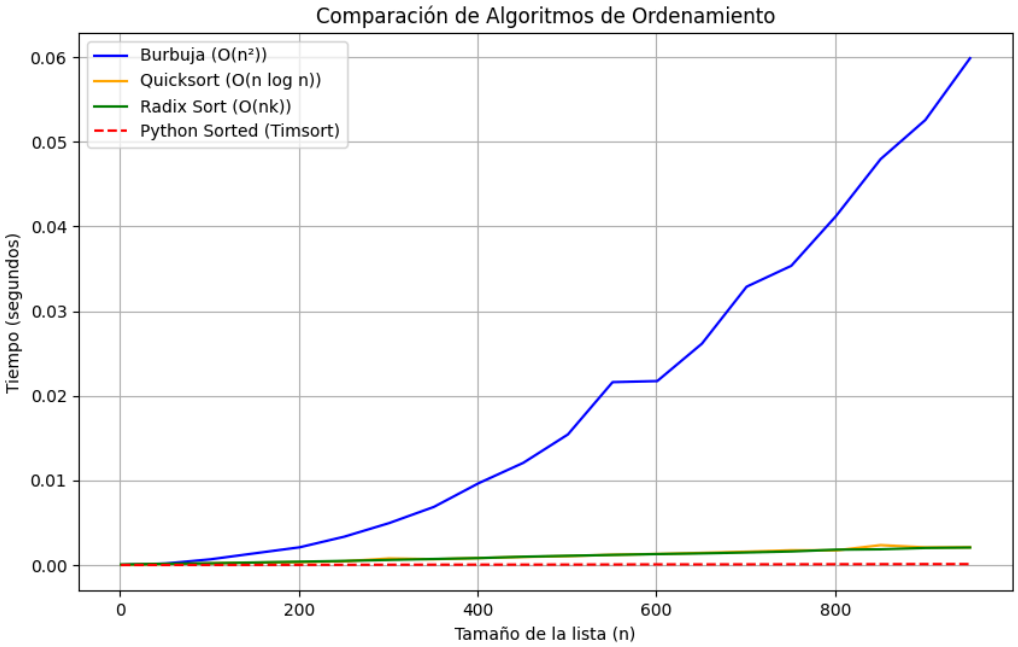
1. **Burbuja:** Implementación clásica con dos bucles anidados. Es un algoritmo *in-place*.

2. **Quicksort:** Se utilizó una versión recursiva con selección de pivote central. Aunque la versión implementada genera nuevas listas (no es *in-place*), facilita la lectura y mantiene la complejidad promedio de O(n log n).
3. **Radix Sort (LSD):** Se implementó la variante *Least Significant Digit*. Utiliza una subrutina de **Counting Sort** para ordenar de forma estable posición por posición, desde las unidades hasta las decenas de mil.
4. **Benchmarking (main.py):** El script genera listas aleatorias, realiza copias para no afectar la medición del siguiente algoritmo y utiliza matplotlib para visualizar la curva de rendimiento.

```
1   import sys
2
3   class Ordenador:
4       """Clase que agrupa los algoritmos de ordenamiento requeridos."""
5
6       @staticmethod
7       def burbuja(lista):···
21
22      @staticmethod
23      def quicksort(lista):···
32
33      @staticmethod
34      def _quicksort_recursivo(lista):···
42
43      @staticmethod
44      def radix_sort(lista):···
58
59      @staticmethod
60      def _counting_sort_for_radix(lista, exp):···
```

C. Resultados y pruebas

La gráfica unificada revela comportamientos consistentes con la teoría asintótica:



- **Burbuja (Azul)**: Presenta una curva parabólica característica de O(n^2). A medida que N aumenta, el tiempo crece drásticamente.
- **Quicksort (Naranja)**: Muestra un crecimiento mucho más controlado. Es significativamente más rápido que Burbuja para valores de N > 100.
- **Radix Sort (Verde)**: Presenta una línea casi recta. Dado que el número de pasadas es fijo (k=5), su rendimiento es excepcional para este tipo de datos numéricos.
- **Python sorted() (Rojo)**: Es el más eficiente de todos. Esto se debe a que utiliza Timsort, un algoritmo híbrido (MergeSort + InsertionSort) escrito en C, altamente optimizado para arquitecturas modernas.

4. Análisis de resultados específicos / Discusión

4.1 Eficiencia de la Estructura: LDE vs. Listas de Python

Una de las discusiones centrales de este trabajo radica en la elección de la **Lista Doblemente Enlazada (LDE)** para el Problema 2. Si bien las listas nativas de Python son extremadamente rápidas por estar implementadas en C, presentan una deficiencia en la operación pop(0) (extracción al inicio), la cual es O(n) debido al desplazamiento de todos los elementos restantes.

En el **Juego de Guerra**, donde las extracciones al inicio son constantes, nuestra LDE demostró ser teóricamente superior al mantener un costo **O(1)**. Esto garantiza que, independientemente de si el mazo tiene 52 o 10,000 cartas, el tiempo de procesamiento de un turno no se degrada.

4.2. El fenómeno de las constantes en el Ordenamiento

Al analizar la gráfica del Problema 3, surge una discusión interesante sobre los algoritmos de orden lineal y logarítmico:

- **Radix Sort vs. Quicksort:** Aunque Radix Sort se percibe casi como una línea recta (eficiencia lineal), para tamaños de lista muy pequeños, Quicksort o incluso sorted() pueden ser competitivos. Sin embargo, al trabajar con números de 5 dígitos fijos, la constante k=5 de Radix Sort lo vuelve imbatible frente al crecimiento de n log n de Quicksort a medida que N se acerca a 1000.
- **Costo de creación de objetos:** Se observó que el método copiar() de la LDE es más lento que invertir(), a pesar de que ambos son O(n). Esto abre la discusión sobre el costo oculto de la **instanciación de objetos**. Mientras invertir solo reasigna referencias, copiar debe solicitar memoria al sistema para cada nuevo nodo, lo que añade un *overhead* significativo que se refleja en la pendiente de la gráfica.

4.3. Implementaciones Nativas vs. Implementaciones Propias

Un hallazgo general del proyecto es la supremacía en tiempos de ejecución de la función sorted(). Esto nos permite discutir la diferencia entre **complejidad algorítmica** y **eficiencia de implementación**: Aunque nuestro **Quicksort** y el **Timsort** (sorted) de Python comparten la categoría O(n log n), la implementación nativa está escrita en C y optimizada para el manejo de caché del procesador. No obstante, el valor de nuestra implementación radica en la **flexibilidad**: nuestra LDE permite el uso de punteros para operaciones específicas que las estructuras de bajo nivel no siempre permiten de forma directa sin incurrir en copias costosas.

5. Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas:

- Gemini (Google)

Descripción del uso:

- **Generación de ideas/estructura:** Se utilizó para estructurar el informe técnico basándose en las rúbricas de la cátedra y para organizar los puntos clave del análisis de complejidad.
- **Asistencia en código/redacción:** Se empleó para mejorar la claridad de los comentarios en el código (Docstrings), asegurar que la redacción del informe tuviera un tono académico adecuado y para depurar la lógica de punteros en métodos específicos como invertir.
- **Consulta teórica:** Se consultó para profundizar en detalles técnicos de algoritmos específicos, como la implementación interna de **Timsort** en Python y las ventajas teóricas de la **Lista Doble Enlazada** frente a listas nativas en operaciones de extremos.

Declaración de responsabilidad: Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las

herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. Trabajo en equipo

6.1. Estimación inicial del tiempo

- **Tiempo total estimado:** 22 horas de trabajo.
- **Distribución planificada:**
  - **Puesta en común e inicio de repositorio:** 2 horas.
  - **Desarrollo del Problema 1 (LDE):** 6 horas.
  - **Desarrollo del Problema 2 y 3:** 8 horas.
  - **Integración, revisión y redacción final:** 6 horas.

6.2. Organización y Roles

El equipo definió roles basados en la especialización de módulos para optimizar los tiempos de entrega:

- **Integrante A (Dinamarca):** Encargada del desarrollo del **Problema 1**. Diseñó la estructura de la ListaDobleEnlazada, implementando los métodos de manipulación de nodos, decoradores @property y validando la eficiencia O(n) de la copia e inversión. Posteriormente, asumió el rol de **editora técnica**, emprolijando el código global y redactando el informe final en Google Docs.
- **Integrante B (Neme Ferrari del Sel):** Responsable de los **Problemas 2 y 3**. Implementó la lógica del Mazo y el motor del JuegoGuerra basándose en la LDE. Desarrolló los algoritmos de ordenamiento (Burbuja, Quicksort, Radix Sort) y el script de benchmarking con matplotlib.

Mecanismos de coordinación:

- **Inicio Presencial:** La organización del repositorio de GitHub y la primera estructura del proyecto se realizaron en una clase presencial, estableciendo las bases del flujo de trabajo.
- **Coordinación Remota:** Debido a la falta de coincidencia horaria posterior, la comunicación se mantuvo fluida por **WhatsApp** para resolver dudas técnicas y coordinar los *commits* al repositorio.
- **Integración de Archivos:** Se utilizó **GitHub** como herramienta central para combinar los archivos de trabajo. La redacción y pulido final se realizó de forma colaborativa mediante un documento compartido en **Google Docs**, facilitando la revisión cruzada.

6.3. Resolución de discrepancias

Las discusiones se centraron en la integración de los módulos:

- **Conexión LDE-Mazo:** Se debatió cómo el Mazo debía interactuar con la lista. Se decidió mantener el encapsulamiento para que el juego no accediera directamente a los nodos, sino a los métodos del TAD.
- **Estilo de código:** Se discutió la unificación de nombres de variables y la inclusión de bloques de compatibilidad para que el proyecto fuera ejecutable de forma universal por los docentes.

6.4. Comparación: Tiempo estimado vs. Tiempo real

- **Tiempo real total:** 28 horas.
- **Diferencias detectadas:**
    1. **Problema 1 (LDE):** La finalización del código en casa llevó más tiempo del previsto.
    2. **Integración y Emprolijamiento:** Esta fase fue la que más se extendió. Unificar el estilo y asegurar que todas las rutas de importación fueran correctas en GitHub requirió algunas revisiones imprevistas.
    3. **Redacción:** La elaboración detallada del informe y la discusión de los resultados de las gráficas demandó una revisión de la teoría para asegurar la práctica académica.

Reflexión: La transición de lo presencial a lo remoto funcionó bien gracias al uso constante de WhatsApp y GitHub. El desvío de tiempo se debió principalmente a la fase de "emprolijamiento" final, confirmando que la integración de partes individuales siempre requiere un margen de tiempo adicional.

7. Conclusiones

El desarrollo de este Trabajo Práctico permitió consolidar los conceptos teóricos de la asignatura mediante su aplicación directa en problemas de lógica compleja y análisis de rendimiento. A modo de cierre, se presentan las siguientes reflexiones:

7.1. Cumplimiento de objetivos

Se cumplieron satisfactoriamente los objetivos planteados al inicio del proyecto. Se logró diseñar e implementar desde cero una **Lista Doblemente Enlazada (LDE)** robusta, que sirvió como motor principal para una simulación de juego funcional. Asimismo, se validó la eficiencia de distintos algoritmos de ordenamiento, logrando una comprensión profunda de las diferencias entre el costo teórico (notación Big O) y el rendimiento real en un entorno de ejecución.

7.2. Dificultades y superación

La principal dificultad técnica radicó en la **gestión de punteros** dentro de la LDE, especialmente en los métodos de inserción y extracción en posiciones arbitrarias. Estos errores lógicos se superaron mediante la técnica de "escritorio" (seguimiento manual de

variables) y la ejecución de tests unitarios que permitieron identificar casos borde, como listas con un solo elemento o extracciones que dejan la lista vacía.

Otra dificultad fue la **coordinación remota** tras el primer encuentro presencial. Esta barrera se superó estableciendo un flujo de trabajo claro basado en la modularización: cada integrante trabajó en archivos separados que luego fueron integrados y validados en conjunto, asegurando que los cambios de uno no afectaran la estabilidad del código del otro.

7.3. Aprendizajes realizados

Más allá de la sintaxis de Python, este trabajo dejó aprendizajes fundamentales en tres áreas:

1. **Pensamiento Algorítmico:** Aprendimos que la elección de una estructura de datos no es trivial; una mala decisión (como usar una lista común para una cola) puede degradar el rendimiento de un sistema a medida que los datos crecen.
2. **Optimización Ética:** El uso de herramientas de IA y la comparación con las funciones nativas del lenguaje (sorted) nos enseñaron que, aunque existen soluciones automáticas y optimizadas, el rol del programador es entender la lógica subyacente para poder personalizar y depurar sistemas complejos.
3. **Gestión de Proyectos:** La experiencia de trabajar con **GitHub**, manejar versiones de código y redactar un informe técnico colaborativo nos brindó una perspectiva real de cómo se desarrolla software en equipo, donde la documentación y la claridad del código son tan importantes como su funcionalidad.

En conclusión, este trabajo práctico no solo fue un ejercicio de programación, sino un entrenamiento en la resolución de problemas y la toma de decisiones técnicas fundamentadas.

8. Referencias y Bibliografía

Para el desarrollo de este proyecto se consultaron fuentes oficiales de documentación técnica y material pedagógico recomendado por la cátedra, garantizando la rigurosidad científica de las implementaciones.

8.1. Bibliografía Principal

- **Miller, B. N., & Ranum, D. L.** *Problemas de resolución con algoritmos y estructuras de datos usando Python*. Recuperado de: Runestone Academy - Pythoned
    - *Uso:* Consulta para la implementación del TAD Lista Doble Enlazada y conceptos de recursividad en Quicksort.

8.2. Documentación Técnica

- **Python Software Foundation.** *Sorting HOW TO*. Recuperado de: Python 3 Documentation
    - *Uso:* Investigación sobre el funcionamiento de **Timsort** y comparativa de eficiencia del método sorted().

8.3. Recursos de la Cátedra

- **LICA - FIUNER.** *Códigos de teoría y práctica para estudiantes*. Recuperado del repositorio oficial de GitHub: FIUNER-LICA/ayed-codigos
    - *Uso:* Obtención de los esqueletos de código para el Juego de Guerra y plantillas de pruebas unitarias (tests).